

TriplePeek: From Entity Search to Live Linked Data Exploration

Albert Khomich

Affiliation to be added, Country to be added

Abstract. Knowledge graphs expose structured information through SPARQL endpoints, yet finding an appropriate resource often requires knowledge of its identifier or the underlying vocabulary. We present TriplePeek, a configurable web interface that connects lexical entity discovery with live Linked Data exploration. A local PostgreSQL catalog stores resource identifiers, labels, and selected searchable metadata. Once a resource is selected, its IRI anchors an endpoint-provided description or an operator-defined SPARQL query. Query templates can delegate federated retrieval to the configured endpoint through SERVICE clauses. The interface combines navigable RDF tables and grouped query values with images and maps when suitable values are available. We describe the catalog lifecycle, ranking procedure, query execution, and a demonstration based on a bundled catalog of 10,000 Wikidata resources. The contribution is an integrated, reusable discovery workflow; comparative retrieval quality, performance, and usability remain to be evaluated.

Keywords: Knowledge graphs · RDF · SPARQL · Entity search

1 Introduction

RDF identifies resources using IRIs and represents their relations as triples [3]. SPARQL provides expressive graph access [4], but users often begin with a name rather than an identifier or knowledge of the vocabulary. They need to find the corresponding resource before inspecting its graph context and the information available from other endpoints.

TriplePeek connects discovery and exploration through a shared IRI. Maintainers select searchable metadata for a local catalog and provide reusable SPARQL queries; users find an entity and open its actions without writing SPARQL. Discovery terminology can be curated independently of graph literals, while query actions remain under the interface maintainer’s control.

The system integrates (i) a catalog-to-IRI-to-query workflow, (ii) catalog generation and transactional import, and (iii) file-based query actions with RDF, media, and location views. This is an interface contribution, not a new indexing or federation algorithm. Source and documentation are available online.¹

¹ <https://github.com/dice-group/triple-peek>; documentation: <https://dice-group.github.io/triple-peek/>.

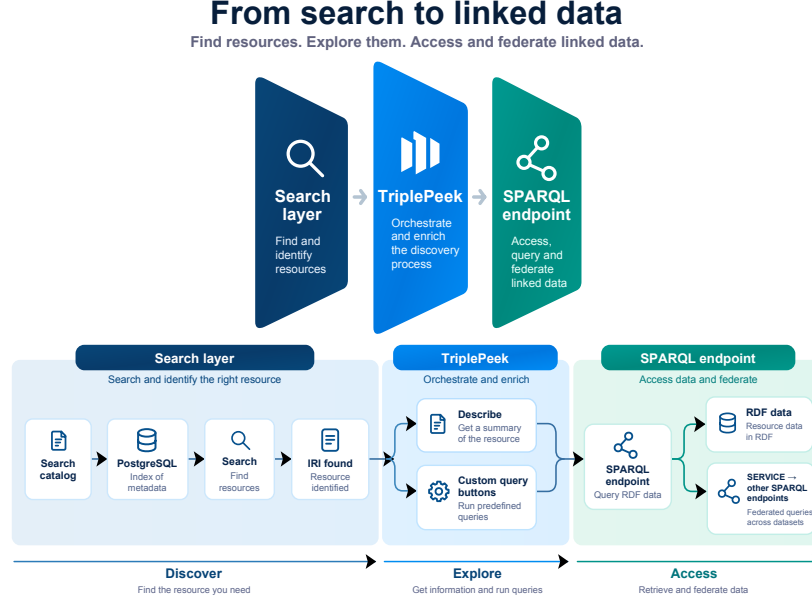


Fig. 1. TriplePeek workflow. Catalog search identifies an IRI; the application then requests a description or instantiates a query template. SERVICE clauses are executed by the configured endpoint. Responses may be RDF graphs, SELECT bindings, or ASK booleans. “Summary” denotes an endpoint-defined DESCRIBE result.

Related work. YASGUI supports SPARQL editing and result visualization [7], whereas TriplePeek starts from catalog search and exposes operator-defined actions. LodView renders navigable HTML views of RDF resources [2]; TriplePeek adds a separately maintained discovery catalog. SemFacet supports semantic faceted search and interactive query refinement [1], while TriplePeek exposes fixed entity-anchored templates. These are differences in interaction models, not evidence of comparative effectiveness.

2 Architecture and Data Flow

Figure 1 shows the architecture. The *search layer* maps text to records; the *application* displays candidates and instantiates actions; the *endpoint* executes queries and any configured federation. The implementation uses TypeScript, Next.js, React, PostgreSQL, N3.js, and MapLibre GL. Docker Compose separates the database, application, catalog generator, and importer.

Let E be the configured endpoint and C a finite catalog:

$$C = \{(u, \ell, \mathbf{m})\}, \quad q \mapsto C_q \mapsto u \mapsto \text{eval}_E(T[u]). \quad (1)$$

Here u is a resource IRI, ℓ its display label, $\mathbf{m}$ selected textual metadata, q a search string, and C_q the ranked candidates. A selected template T is instantiated with u . The built-in action uses `DESCRIBE <u>`. Its returned graph is determined by the endpoint; it is not guaranteed to contain every incident triple.

The catalog is a discovery projection, not an RDF replica. Metadata may combine graph values, alternative terminology, and curated descriptions; its coverage bounds text-search recall. Search requests access PostgreSQL only. Opening an action triggers SPARQL retrieval, so search input changes do not contact the endpoint. This separates workloads without establishing a latency improvement.

2.1 Catalog Preparation and Import

A CSV catalog contains at least `iri` and `label`; further columns become searchable metadata. Alternatively, the generator executes an operator-provided SELECT query against E and exports bindings using LIMIT/OFFSET pagination. Operators can bound the number of exported rows or the UTF-8 file size. The generator writes complete CSV rows into a temporary file and replaces the destination only after successful export. Stable pagination requires a suitably ordered query and a sufficiently stable endpoint dataset; the bundled query does not enforce either condition.

Before importing, the pipeline normalizes the second column to `label` and validates headers, CSV structure, and IRIs. Duplicate IRIs trigger grouping of distinct, nonempty field values, followed by revalidation. A PostgreSQL staging table holds validated rows. Schema changes, index creation, and insertion occur within one transaction, so an import failure leaves the search table unchanged. File normalization occurs separately and can modify the CSV.

By default, an IRI already present in the table causes insertion to fail. With `--update`, incoming rows replace searchable values for the corresponding IRIs; omitted metadata columns become NULL and absent IRIs remain unchanged. The importer creates a full-text GIN index, a trigram GIN index, and an index on lower-case labels for prefix matching. All imported fields, including the IRI, contribute to the generated search text.

2.2 Entity Search and Ranking

Candidate retrieval combines exact labels, label prefixes, full-text matching, and trigram word similarity. Let x_c be the concatenated fields of a record c and v_c its PostgreSQL text-search vector. The priority of a matching record is the first satisfied condition:

$$p(c, q) = \begin{cases} 1 & \text{case-insensitive exact label,} \\ 2 & \text{case-insensitive literal label prefix,} \\ 3 & \text{full-text match against } v_c, \\ 4 & \text{trigram word-similarity match against } x_c. \end{cases} \quad (2)$$

Full-text matching uses `plainto_tsquery` with the `simple` configuration. Within priorities 1–3, the score is `ts_rank_cd`; within priority 4, it is `word_similarity` [5,6]. Trigram eligibility uses PostgreSQL’s configured word-similarity threshold. No learned model or embedding search participates in this ranking.

Results are ordered by ascending priority, descending score, lower-case label, original label, and IRI. A keyset cursor records this ordering tuple and the query string. The API returns at most 20 records per page and deduplicates by IRI before limiting. The interface waits 300 ms after an input change before requesting a page. Result cards show the label, IRI, and bounded metadata summaries; metadata too long to display can still contribute to search.

2.3 Query Actions and Presentation

Each regular `.sparql` file in the application’s buttons directory defines an action. Templates use the literal placeholder `${iri}` inside an IRI reference. For example, the following entity-summary action retrieves an image and coordinates:

```
PREFIX wdt: <http://www.wikidata.org/prop/direct/>
SELECT ?image ?coord WHERE {
  OPTIONAL { <${iri}> wdt:P18 ?image }
  OPTIONAL { <${iri}> wdt:P625 ?coord }
}
```

The server validates the supplied IRI, substitutes all placeholders, and recognizes `SELECT`, `ASK`, `CONSTRUCT`, or `DESCRIBE` after prefix/base declarations. Templates are trusted operator configuration; update forms are unsupported. Requests use the configured endpoint and a 15-second timeout. `SELECT` and `ASK` request SPARQL JSON results; graph queries prefer Turtle. A template may include `SERVICE` clauses, whose execution and remote-source access are delegated to the endpoint [4].

Parseable RDF graph responses are displayed as subject–predicate–object tables, with a graph column where applicable. Selecting a named RDF term reruns the panel’s action for that IRI, and a local history supports backward navigation. `SELECT` results are grouped into distinct values per projected variable, and `ASK` results appear as booleans. Grouping suits entity attributes but does not preserve correlations between variables across result rows. Recognized image URLs and point coordinates add media previews and map markers; these views are heuristic interpretations of returned values.

3 Demonstration and Evidence

Scenario. The bundled Wikidata catalog contains 10,000 rows and 10,000 distinct IRIs with four fields: IRI, label, type label, and country. These counts were obtained directly from the distributed CSV. Its generation query selects

castles with English labels, an image, and coordinates, enriching them with type and country labels. A visitor searches for *Neuschwanstein Castle*, identifies the resource, and opens Describe to inspect its graph. The Details action requests an image, coordinates, and an English description. The Embedding action uses SERVICE to retrieve a precomputed Keci embedding from a separate endpoint. It demonstrates cross-source retrieval, not embedding generation or similarity search. Live results depend on the configured services and may differ from catalog metadata.

Conference deployment. The proposed demonstration runs the application and PostgreSQL on a laptop with a preloaded catalog. Visitors can search, inspect RDF terms, open action panels, and follow geographic or image views. A short configuration walkthrough then shows how replacing the CSV and adding a query file adapts the interface to another dataset. Internet access is needed for live endpoint queries and external media. The conference plan includes preparing a recorded walkthrough and captured example responses as an explicitly labeled fallback for service outages.

Evidence and evaluation scope. The bundled regression suite checks query-file discovery, supported query forms, IRI substitution, typed responses, and error handling using mocked endpoint responses. All eight scenario subtests passed in a local execution. This suite does not assess database ranking, public-endpoint interoperability, or user experience. The present contribution is therefore a system demonstration supported by implementation analysis and a concrete catalog. A subsequent evaluation should measure retrieval quality with relevance judgments, search and action latency separately, endpoint-request counts, and task completion with representative users.

4 Limitations and Ethical Use of Data

Catalog updates are explicit; no synchronization mechanism guarantees agreement with live RDF. Recall is bounded by the chosen resources and terms. Duplicate grouping is performed in memory, so streaming validation does not imply bounded memory for the whole pipeline. The primary endpoint must support any SERVICE clauses in the templates. Large responses are processed in memory, SELECT displays discard tuple associations, and the interactive RDF parser does not handle JSON-LD or RDF/XML. These constraints motivate bounded, entity-summary queries and further interoperability testing.

The example uses public Wikidata metadata. No human-participant study or personal search-log collection is reported. Deployments must consider data and image licenses, provenance, endpoint policies, and the suitability of external services. Future user studies should obtain appropriate consent and minimize retained interaction data.

5 Conclusion

TriplePeek connects a curated lexical catalog to live, IRI-anchored SPARQL exploration. Its configurable metadata and query actions let maintainers adapt the discovery workflow while leaving RDF execution with existing endpoints. The prototype supports a concrete demonstration of this separation; systematic retrieval, performance, and usability evaluation are the next steps.

References

1. Arenas, M., Grau, B.C., Kharlamov, E., Marciuška, Šarūnas., Zheleznyakov, D., Jimenez-Ruiz, E.: SemFacet: Semantic faceted search over Yago. In: Proceedings of the 23rd International Conference on World Wide Web (Companion). pp. 123–126. Association for Computing Machinery, New York, NY, USA (2014). <https://doi.org/10.1145/2567948.2577011>, <https://www.cs.ox.ac.uk/isg/tools/SemFacet/SemFacetwww14.pdf>
2. Camarda, D.V., Antonuccio, A.: LodView: IRI dereferencer, RDF to HTML. Software repository, <https://github.com/LodLive/LodView>, accessed 24 September 2026
3. Cyganiak, R., Wood, D., Lanthaler, M.: RDF 1.1 concepts and abstract syntax. W3c recommendation, World Wide Web Consortium (Feb 2014), <https://www.w3.org/TR/2014/REC-rdf11-concepts-20140225/>, editors; 25 February 2014
4. Harris, S., Seaborne, A.: SPARQL 1.1 query language. W3c recommendation, World Wide Web Consortium (Mar 2013), <https://www.w3.org/TR/2013/REC-sparql11-query-20130321/>, editors; 21 March 2013
5. PostgreSQL Global Development Group: PostgreSQL 16 Documentation: Controlling Text Search, <https://www.postgresql.org/docs/16/textsearch-controls.html>, accessed 24 September 2026
6. PostgreSQL Global Development Group: PostgreSQL 16 Documentation: pg_trgm, <https://www.postgresql.org/docs/16/pgtrgm.html>, accessed 24 September 2026
7. Rietveld, L., Hoekstra, R.: The YASGUI family of SPARQL clients. *Semantic Web* **8**(3), 373–383 (2017). <https://doi.org/10.3233/SW-150197>, <https://doi.org/10.3233/SW-150197>